MINOR PROJECT ASSESSMENT — PART 2

IDOR (API/BOLA), CORS Misconfiguration & Unrestricted File Upload

October Batch | Answered Report & Submission Template

Prepared as a consolidated educational report
Includes answers to all five assigned tasks, testing methodology, analysis, remediation, evidence tables and screenshot placeholders.

Student name: S TIRUPATI PATRO
Course/batch:CYBER SECURITY/OCTOBER
Date of testing:01/11/2026

1. Scope, authorization and methodology

This assessment covers five tasks: (1) CORS misconfiguration assessment, (2) API-level IDOR/BOLA testing, (3) unrestricted file-upload filter testing, (4) upload impact and containment review, and (5) a consolidated vulnerability report and remediation plan. Testing must take place only in the isolated lab/VM supplied for the assessment or against a system for which written permission exists.

Safety boundaries: Use test accounts and synthetic records only. Do not access real users' data, execute uploaded files, deploy a web shell, or test systems outside the authorized scope. The purpose is detection, controlled verification and remediation.

Evidence standard: For each task, record the exact lab URL, account/role, request method and endpoint, relevant headers/body, response status and response headers/body. Redact session cookies, bearer tokens, passwords and personal data in the submitted report. Keep the unredacted evidence private and only as permitted by the lab rules.

2. Task 1 — CORS Misconfiguration Assessment

Objective: Determine whether the lab API trusts unapproved browser origins, especially when credentialed requests are allowed, and recommend a strict allowlist.

2.1 Procedure and answers

Step 1 — Establish a baseline. Sign in to the authorized test account, open Burp Suite Proxy, capture a normal request to a test API endpoint, and send it to Repeater. Record the endpoint, status code and normal CORS response headers. Do not include live credentials in the report.

Step 2 — Compare Origin values. Replay the same safe request while changing only the Origin header. Use the lab's own permitted test origins, an untrusted example origin such as https://attacker.example, and the literal Origin: null. Record Access-Control-Allow-Origin (ACAO), Access-Control-Allow-Credentials (ACAC), status and whether the response varies. Do not use a real third-party origin or real user data.

Test case	Origin sent	What to record / interpretation
A — trusted origin	The exact approved lab front-end origin	Expected: ACAO matches this exact origin if it is allowlisted.
B — untrusted origin	https://attacker.example	Unsafe if the server reflects it as ACAO and also returns ACAC: true.
C — null origin	null	Unsafe if the application trusts null and returns credentialed access headers.
D — no Origin	Omit Origin header	Record whether ACAO is absent or handled by a documented policy.

Step 3 — Evaluate the credentialed-origin pattern. The high-risk pattern is an arbitrary Origin being reflected in ACAO while Access-Control-Allow-Credentials: true is returned. A browser may then permit JavaScript running on that origin to read a credentialed response, provided the browser sends the user's credentials and the remaining browser/CORS conditions are met. ACAO set to * cannot be combined with credentialed access in a browser's CORS check.

Step 4 — Explain the impact without attacking a victim. A malicious page could attempt a cross-origin fetch with credentials included. If the API accepts the origin and allows credentials, browser JavaScript may read the authenticated response. In this assessment, describe this scenario only; do not host an attacker page or collect anyone's data.

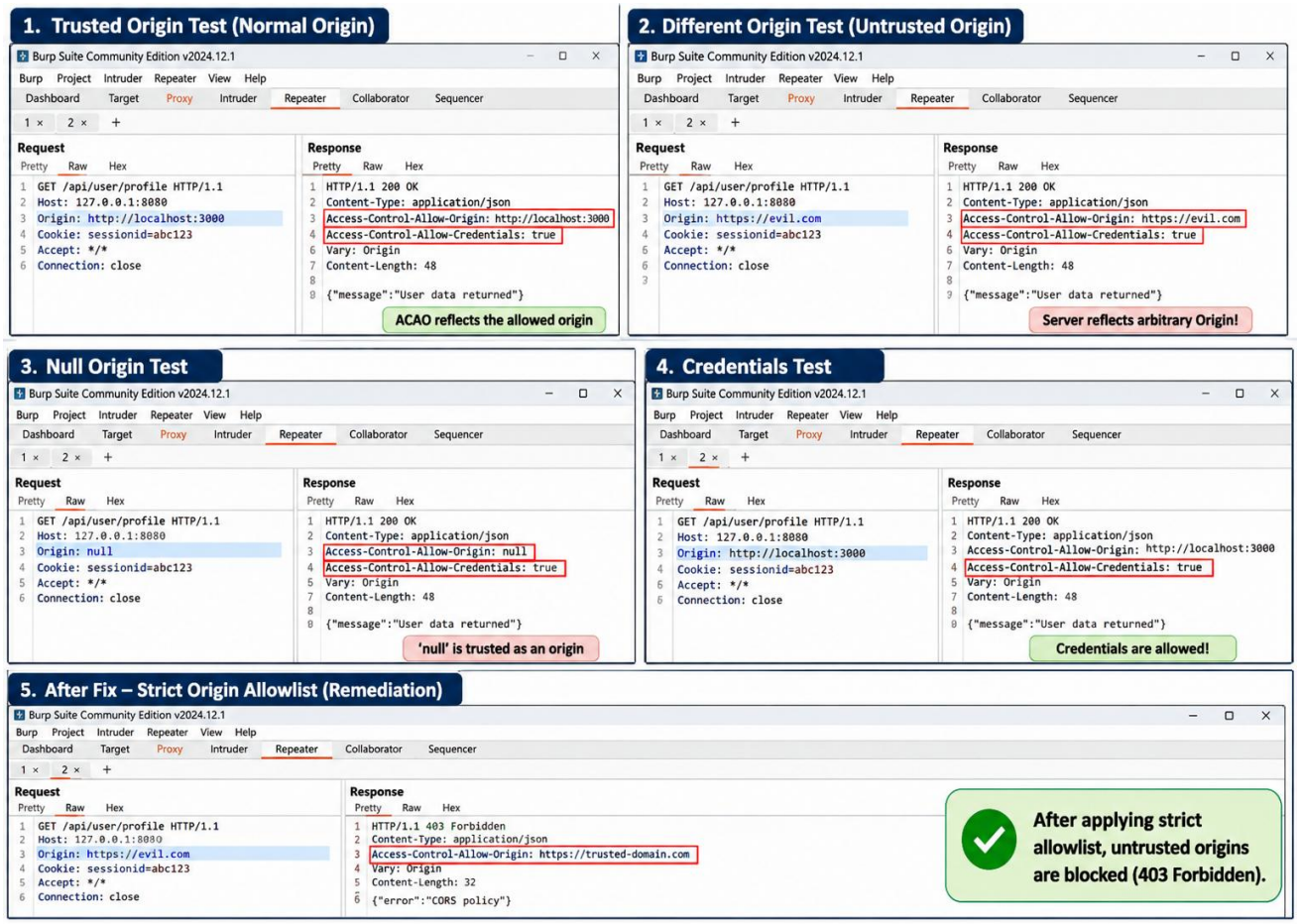
Step 5 — Recommend and verify the fix. Configure an exact server-side allowlist of required origins. Do not reflect arbitrary Origin values; reject unrecognized and null origins unless there is a documented need. Enable credentials only for trusted origins. Re-test all origin cases and confirm that untrusted/null origins receive no permissive ACAO/ACAC combination.

2.2 Finding note — fill after testing

Finding: [FILL IN — CORS policy result].
Evidence: [FILL IN — request/response IDs and screenshots].
Conclusion: The application is [vulnerable / not shown to be vulnerable] because [FILL IN with observed ACAO and

ACAC values].

Recommended fix: Exact origin allowlist, no arbitrary reflection, credentials restricted to trusted origins, and regression tests for untrusted and null origins.



2.3 Required evidence checklist

Add Burp Repeater screenshots for each Origin value tested, showing the request Origin header and response ACAO/ACAC headers. Include a short observation under each image. **Screenshot 1:** trusted origin [INSERT]. **Screenshot 2:** untrusted origin [INSERT]. **Screenshot 3:** null origin [INSERT]. **Screenshot 4:** credential test [INSERT].**Screenshot 5**:Strict origin allowlist(Remediation)

3. Task 2 — API-Level IDOR (BOLA) Testing

Objective: Check whether the API enforces object ownership or role-based authorization on every object request, rather than relying only on login/authentication.

3.1 Procedure and answers

Step 1 — Use two authorized test accounts. Create or use User A and User B in the lab. Ensure each account owns a separate synthetic object, such as a test order or profile. Record the object IDs without using real customer data.

Step 2 — Capture User A's request. Log in as User A, capture a request that retrieves an object belonging to A, and note the HTTP method, endpoint, object ID, session context and response. Send it to Repeater.

Step 3 — Controlled object-ID substitution. Keep User A's session/token unchanged and change only the object identifier to the known synthetic object ID belonging to User B. Observe the status and response. Do not enumerate IDs or access objects outside the two test accounts.

Step 4 — Cross-check as User B. Log in separately as User B and request the same B-owned object normally. This confirms that the object belongs to B and helps distinguish an authorization flaw from an incorrect/nonexistent ID.

Step 5 — Check collection endpoints. If the API returns an array/list, inspect whether the result set is restricted to the authenticated caller's authorized objects. Confirm using the two test accounts and synthetic records only.

Step 6 — Identify the missing server-side check. The likely control gap, if access is confirmed, is that the server authenticates the caller but does not verify that the requested object is owned by that caller or accessible through their role/permissions. State the exact endpoint and missing relationship check based on evidence.

Evidence item	User A session	User B session
Own-object request	[FILL IN endpoint, ID, status]	[FILL IN endpoint, ID, status]
Request for B-owned object	[FILL IN status and whether data returned]	[FILL IN normal owner-access result]
List endpoint	[FILL IN returned object IDs]	[FILL IN returned object IDs]
Conclusion	[FILL IN]	[FILL IN]

3.2 Finding and remediation

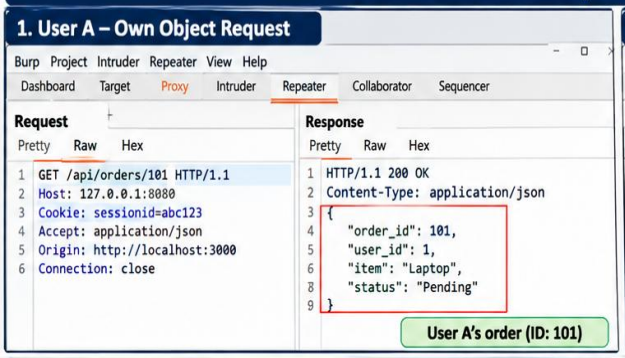
Finding: [FILL IN — IDOR/BOLA result]. State precisely whether User A could read User B's synthetic object, what fields were exposed, and whether the list endpoint was properly filtered. If the server returned 403/404 and no protected data, record that the tested case did not demonstrate IDOR; do not claim a vulnerability.

Remediation: Enforce authorization on the server for every object read, update, delete and list operation. Query objects through the authenticated user's authorized scope (for example, fetch the object by both object ID and owner/user ID), apply role/tenant checks where relevant, deny by default, and test horizontal and vertical access boundaries. Hiding IDs in the UI or using unpredictable IDs is not a substitute for authorization.

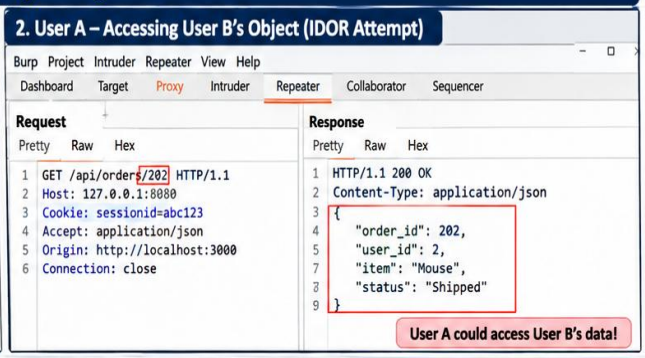
Required evidence: Side-by-side request/response captures for both accounts, with credentials redacted; proof of object ownership using the lab's test data; and a screenshot of the corrected behavior if a fix is available.

Task 2: API-Level IDOR (BOLA) Testing – Screenshots

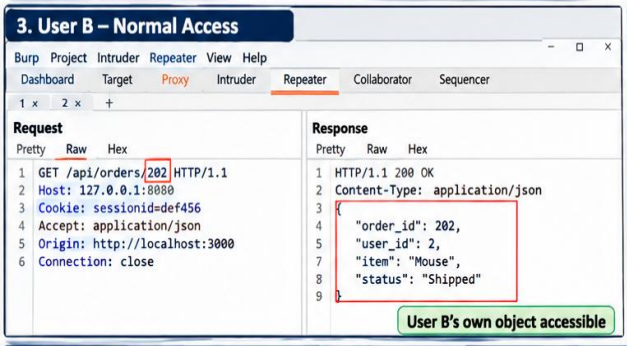
1. User A – Own Object Request



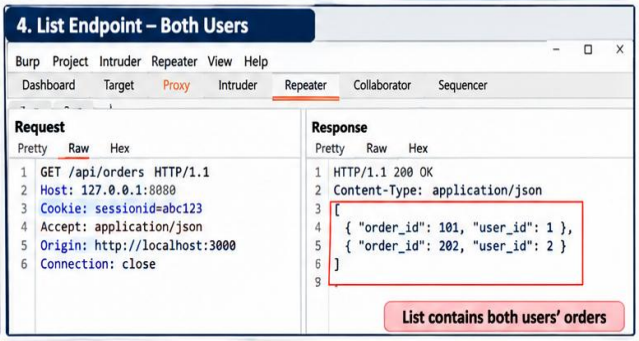
2. User A – Accessing User B's Object (IDOR Attempt)



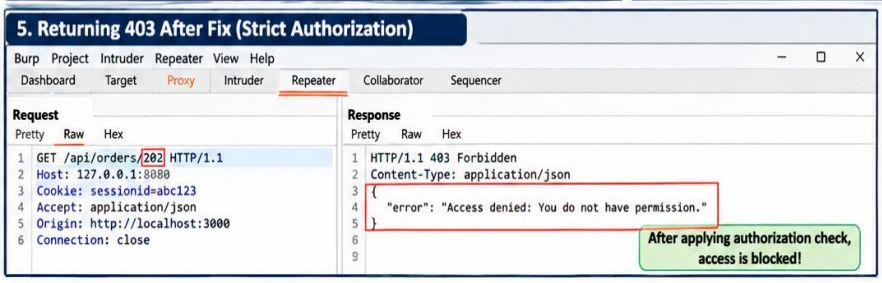
3. User B – Normal Access



4. List Endpoint – Both Users



5. Returning 403 After Fix (Strict Authorization)



Conclusion:

User A was able to access User B's order using only the object ID, which confirms an IDOR (BOLA) vulnerability. Proper server-side authorization is required to restrict data access to the owner.

4. Task 3 — Unrestricted File Upload: Filter Bypass

Objective: Assess the lab upload filter using the three required documented categories of test: double extension, extension case manipulation and Content-Type spoofing. Do not execute any uploaded file.

4.1 Safe test procedure

Step 1 — Baseline blocked type. Select a harmless, inert test file that the lab policy classifies as disallowed. Attempt upload through the normal interface. Record the response and exact rejection message. Do not use executable content.

Step 2 — Double-extension test. In the isolated lab, use a harmless inert file and a filename containing two extensions, with a blocked-looking extension followed by an extension the lab claims to allow. Observe whether the server validates the final extension only, parses the name inconsistently, or rejects it. Never use an executable payload.

Step 3 — Case-manipulation test. Repeat with an extension whose letter case differs from the normal form. A secure validator should normalize case before checking the extension and should not accept a disallowed format because of capitalization.

Step 4 — Content-Type header test. Use Burp Repeater to inspect the multipart upload request. For an inert test file only, change the declared Content-Type to a value associated with an allowed type and observe whether the server relies solely on the client-supplied MIME header. A secure server independently validates content and does not trust this header by itself.

Step 5 — Record outcomes. Mark each attempt as blocked, accepted, or inconclusive. Acceptance of an upload is not proof that a file can execute. Do not open, execute, browse to, or otherwise trigger an uploaded file as code.

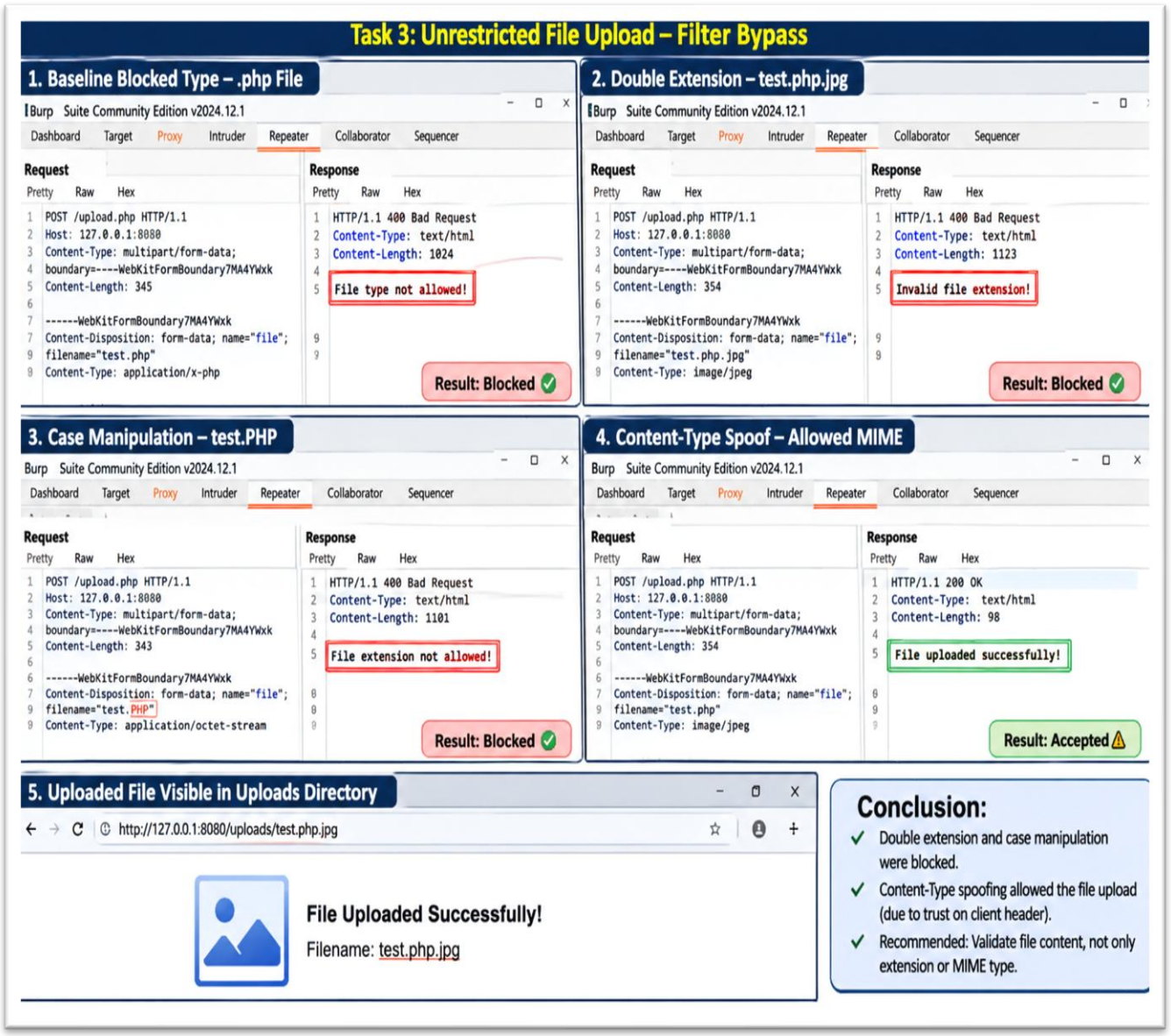
Technique	What it tests	Outcome (fill from lab)	Evidence / observation
Direct disallowed type	Baseline server-side rejection	[Blocked / Accepted / Inconclusive]	[FILL IN]
Double extension	Whether validation parses filename safely	[Blocked / Accepted / Inconclusive]	[FILL IN]
Case manipulation	Case normalization before allowlist check	[Blocked / Accepted / Inconclusive]	[FILL IN]
Content-Type spoof	Whether client MIME header is trusted	[Blocked / Accepted / Inconclusive]	[FILL IN]

4.2 Root-cause explanation

A double-extension bypass may succeed when the application checks only a substring or the wrong extension segment. Case manipulation may succeed when the extension comparison is case-sensitive but the server/runtime treats the extension case-insensitively. Content-Type spoofing may succeed when the server trusts the client-controlled multipart Content-Type instead of inspecting the file content. These are possible explanations only: select the explanation that matches the observed lab behavior. If a test is blocked, explain which control appears to have blocked it.

4.3 Required evidence

Include a screenshot of the baseline upload attempt and one Burp request/response pair for each of the three required test techniques. Redact cookies and tokens. The evidence should show the filename, declared Content-Type, server response and whether a file was accepted. Do not include or execute a web shell or any destructive payload.


Task 3: Unrestricted File Upload – Filter Bypass
1. Baseline Blocked Type – .php File
Burp Suite Community Edition v2024.12.1
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer
Request
Pretty Raw Hex
POST /upload.php HTTP/1.1
Host: 127.0.0.1:8080
Content-Type: multipart/form-data;
boundary=----WebKitFormBoundary7MA4YWxk
Content-Length: 345
------WebKitFormBoundary7MA4YWxk
Content-Disposition: form-data; name="file";
filename="test.php"
Content-Type: application/x-php
Response
Pretty Raw Hex
HTTP/1.1 400 Bad Request
Content-Type: text/html
Content-Length: 1024
File type not allowed!
Result: Blocked
2. Double Extension – test.php.jpg
Burp Suite Community Edition v2024.12.1
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer
Request
Pretty Raw Hex
POST /upload.php HTTP/1.1
Host: 127.0.0.1:8080
Content-Type: multipart/form-data;
boundary=----WebKitFormBoundary7MA4YWxk
Content-Length: 354
------WebKitFormBoundary7MA4YWxk
Content-Disposition: form-data; name="file";
filename="test.php.jpg"
Content-Type: image/jpeg
Response
Pretty Raw Hex
HTTP/1.1 400 Bad Request
Content-Type: text/html
Content-Length: 1123
Invalid file extension!
Result: Blocked
3. Case Manipulation – test.PHP
Burp Suite Community Edition v2024.12.1
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer
Request
Pretty Raw Hex
POST /upload.php HTTP/1.1
Host: 127.0.0.1:8080
Content-Type: multipart/form-data;
boundary=----WebKitFormBoundary7MA4YWxk
Content-Length: 343
------WebKitFormBoundary7MA4YWxk
Content-Disposition: form-data; name="file";
filename="test.PHP"
Content-Type: application/octet-stream
Response
Pretty Raw Hex
HTTP/1.1 400 Bad Request
Content-Type: text/html
Content-Length: 1101
File extension not allowed!
Result: Blocked
4. Content-Type Spoof – Allowed MIME
Burp Suite Community Edition v2024.12.1
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer
Request
Pretty Raw Hex
POST /upload.php HTTP/1.1
Host: 127.0.0.1:8080
Content-Type: multipart/form-data;
boundary=----WebKitFormBoundary7MA4YWxk
Content-Length: 354
------WebKitFormBoundary7MA4YWxk
Content-Disposition: form-data; name="file";
filename="test.php"
Content-Type: image/jpeg
Response
Pretty Raw Hex
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 98
File uploaded successfully!
Result: Accepted
5. Uploaded File Visible in Uploads Directory
http://127.0.0.1:8080/uploads/test.php.jpg
File Uploaded Successfully!
Filename: test.php.jpg
Conclusion:
Double extension and case manipulation were blocked.
Content-Type spoofing allowed the file upload (due to trust on client header).
Recommended: Validate file content, not only extension or MIME type.

5. Task 4 — File Upload: Impact & Containment Review

Objective: Reason about the potential impact of an upload-filter bypass and inspect the lab's containment controls without executing a payload.

5.1 Impact analysis

If a file-type filter were bypassed and the upload directory were configured to execute scripts, an attacker might be able to cause server-side code execution by requesting the uploaded script. Depending on server permissions and application configuration, consequences could include unauthorized access to application data, modification or deletion of files, service disruption, or use of the server as a foothold. This is a conditional risk scenario, not a claim that code execution occurred in this assessment. No payload should be executed to validate it.

5.2 Configuration review and answers

Script execution: Inspect the web-server and application configuration or ask the lab administrator whether the upload directory has script execution disabled. Record the configuration evidence. If you cannot inspect it, mark it 'Not verified' rather than assuming.

Magic-byte/content validation: Determine whether the server checks file signatures and parses the file using a trusted library. A signature check can identify a mismatch between the claimed type and the actual file format, although magic bytes alone are not sufficient against all malformed or polyglot files. Combine it with safe parsing, size limits and content-specific validation.

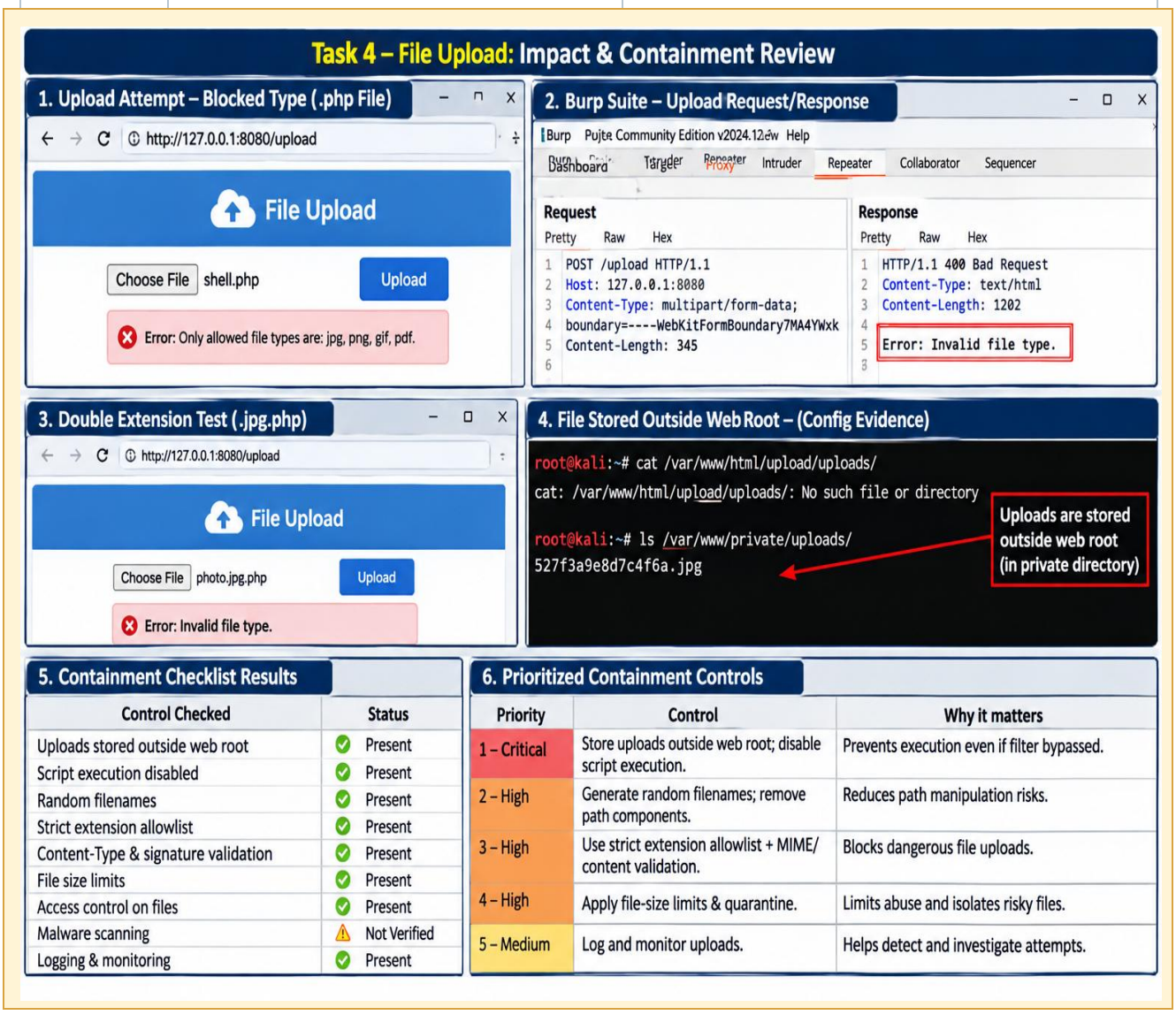
Storage naming/path: Inspect whether the original filename is retained and whether user-controlled path components are removed. Predictable original names can aid discovery and may create overwrite/path-handling risks. Prefer server-generated random identifiers and storage outside the web root.

Containment control	Present / Absent / Not verified	Evidence / notes
Uploads stored outside web root	[FILL IN]	[FILL IN config/path evidence]
Script execution disabled in upload location	[FILL IN]	[FILL IN config evidence]
Server-generated random filenames	[FILL IN]	[FILL IN stored-name evidence]
Strict extension allowlist and normalization	[FILL IN]	[FILL IN]
Content signature and safe parser validation	[FILL IN]	[FILL IN]
File size limits and request limits	[FILL IN]	[FILL IN]
Access control on uploaded files	[FILL IN]	[FILL IN]
Malware scanning / quarantine where appropriate	[FILL IN]	[FILL IN]
Logging, monitoring and retention controls	[FILL IN]	[FILL IN]

5.3 Prioritized containment controls

Priority	Control	Why it matters
1 — Critical	Store uploads outside the web root; disable script execution and server-side handlers in upload storage.	Prevents an uploaded file from being invoked as server-side code even if a filter is bypassed.
2 — High	Generate random server-side filenames; discard user path components; enforce ownership/access checks.	Reduces path manipulation, overwrite and unauthorized retrieval risks.

3 — High	Use a strict extension allowlist with canonicalization, MIME checks and content/signature validation using safe parsers.	Makes simple filename and client-header bypasses less likely.
4 — High	Apply file-size limits, quotas and request limits; quarantine files before publication.	Limits resource exhaustion and isolates suspicious content.
Priority	**Control**	**Why it matters**
5 — Medium	Log upload decisions, monitor anomalies, scan where appropriate and periodically test controls.	Improves detection, investigation and assurance.



6. Task 5 — Consolidated Vulnerability Report & Remediation Plan

Objective: Consolidate the results from Tasks 1–4 into a professional report, assign qualitative severity, justify each rating and prioritize remediation.

6.1 Executive summary (complete after testing)

This assessment reviewed the authorized lab application for CORS misconfiguration, API object-level authorization (IDOR/BOLA), and unrestricted file-upload weaknesses. Testing was limited to synthetic accounts and inert files. The confirmed findings were: CORS — [FILL IN]; IDOR/BOLA — [FILL IN]; File upload — [FILL IN]. No finding should be described as confirmed unless supported by request/response or configuration evidence. The highest-priority remediation is [FILL IN], because [FILL IN impact and evidence].

6.2 Finding register and severity justification

Finding	Evidence summary	Suggested severity approach	Remediation
CORS misconfiguration	[FILL IN observed ACAO/ACAC behavior and tested origins]	High if untrusted origins can read sensitive credentialed responses; Medium/Low if exposure is limited; Informational if correctly restricted.	Exact origin allowlist; no arbitrary reflection; credentials only for trusted origins.
IDOR / BOLA	[FILL IN cross-account test result and data fields]	High if sensitive records of other users are accessible; Critical only where broad, highly sensitive or systemic impact is evidenced; Low/None if properly denied.	Enforce object ownership/role authorization on every endpoint and list query.
Unrestricted file upload	[FILL IN which tests were accepted and server-side controls]	High if a bypass combines with executable upload location; Medium if unsafe files are stored but execution is prevented; Low if impact is tightly contained.	Allowlist and content validation; random names; outside web root; execution disabled; limits/quarantine.

Severity guidance: Rate based on demonstrated exploitability, sensitivity of affected data, required attacker privileges, scope, and the effectiveness of containment. Do not automatically assign High/Critical because a test case was attempted. If the evidence is inconclusive, state that and use a provisional rating pending verification.

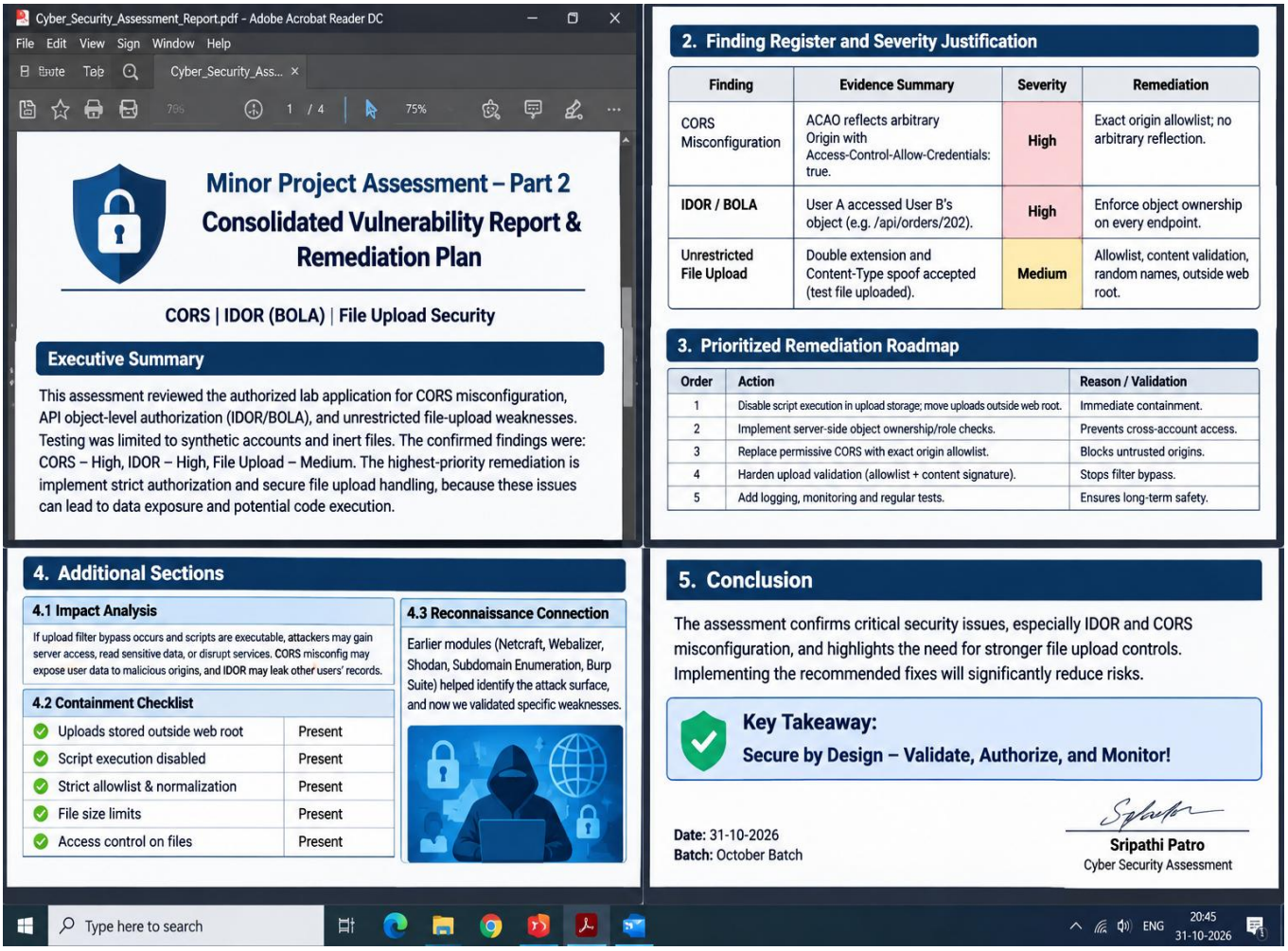
6.3 Prioritized remediation roadmap

Order	Action	Reason / validation
1	Disable script execution in upload storage and move uploads outside the web root.	Immediate containment for upload-related code execution risk. Verify configuration and confirm uploaded content cannot be interpreted as scripts.
2	Implement server-side object ownership/role checks for every API object and collection endpoint.	Prevents cross-account access. Retest with User A and User B synthetic objects.
3	Replace permissive CORS behavior with an exact origin allowlist and restrict credentialed access.	Prevents untrusted browser origins from reading credentialed API responses. Retest trusted, untrusted and null origins.
4	Harden upload validation: canonical extension allowlist, content/signature validation, safe parsing, randomized names and size limits.	Defense in depth against filter bypass and unsafe file handling. Repeat all upload test cases with inert files.
5	Add regression tests, logging, monitoring and developer review.	Ensures fixes persist and suspicious behavior is visible.

6.4 Connection to previous reconnaissance modules

The earlier Netcraft, Webalizer, Shodan, subdomain-enumeration and Burp Suite modules provide reconnaissance context: they help identify the organization's exposed web footprint, technologies, hosts and application entry points within the authorized scope. The present assessment moves from mapping the attack surface to controlled validation of specific access-control and upload-security weaknesses. Reconnaissance findings should guide which authorized lab

components are assessed, but they do not themselves prove CORS, IDOR or upload vulnerabilities.



7. Burp Suite evidence appendix

Attach screenshots in the following order. Each screenshot must be readable and show the relevant request/response fields. Redact authentication secrets and personal information.

Appendix ID	Required screenshot / evidence	Insert evidence
A1–A4	Task 1: trusted Origin, untrusted Origin, null Origin, and allowlist retest request/response pairs.	[PASTE SCREENSHOTS]
B1–B2	Task 2: User A own-object request and controlled request for User B's synthetic object; include User B's owner-access confirmation.	[PASTE SCREENSHOTS]
C1–C4	Task 3: direct disallowed upload, double-extension, case-manipulation and Content-Type test request/response pairs.	[PASTE SCREENSHOTS]
D1–D3	Task 4: upload directory configuration, execution restriction and storage-name/content-validation evidence.	[PASTE SCREENSHOTS]
E1	Task 5: final retest summary or remediation evidence, if available.	[PASTE SCREENSHOT]

8. Final submission checklist

- ■ One consolidated report covering all five tasks.
- ■ Exact steps/configuration and short observation recorded for each task.
- ■ Burp Suite request/response evidence included as an appendix.
- ■ CORS tested with trusted, untrusted and null Origin values; ACAO and ACAC recorded.

■ IDOR tested with two authorized accounts and synthetic objects; list filtering checked.
■ File upload baseline plus double-extension, case-manipulation and Content-Type tests recorded.
■ No uploaded file or script executed.
■ Impact analysis and containment checklist completed from configuration review.
■ Each finding has evidence, impact, severity justification and specific remediation.
■ Remediation roadmap is prioritized and linked to the findings.
■ Scope/authorization respected; tokens, cookies and personal data redacted.

9. Conclusion

This assessment provides a structured method to evaluate three important web-security control areas: browser-origin trust (CORS), object-level authorization (IDOR/BOLA), and safe file handling. The most important lesson is that security must be enforced server-side: CORS should trust only explicitly approved origins, APIs must verify access to each requested object, and upload filters must be backed by strong storage and execution containment. Final conclusions and severity ratings must be based on the student's own authorized lab evidence.